



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Gestión *offline* del banco
de preguntas de Moodle.



Presentado por Miguel Alonso Alonso
en Universidad de Burgos - 7 de Junio
de 2026

Tutor: Dr. César Ignacio García Osorio



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



El Dr. César Ignacio García Osorio, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos

Expone:

Que el alumno D. Miguel Alonso Alonso, con DNI 71482301A, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “Gestión offline del banco de preguntas de Moodle”.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 7 de junio de 2026

Vº Bº del Tutor:

Dr. César Ignacio García Osorio



ÍNDICE DE CONTENIDOS

1 - INTRODUCCIÓN	6
2 - OBJETIVOS DEL PROYECTO	8
2.1 - OBJETIVOS FUNCIONALES	8
2.2 - OBJETIVOS TÉCNICOS	9
2.3 - OBJETIVOS DE APRENDIZAJE PERSONAL	10
3 - CONCEPTOS TEÓRICOS	11
3.1 - MOODLE Y LOS BANCOS DE PREGUNTAS	11
3.2 - TIPOS DE PREGUNTA DE MOODLE	11
3.2.1 - OPCIÓN MÚLTIPLE (MULTICHOICE)	11
3.2.2 - VERDADERO/FALSO (TRUEFALSE)	12
3.2.3 - RESPUESTA CORTA (SHORTANSWER)	12
3.2.4 - NUMÉRICA (NUMERICAL)	12
3.2.5 - EMPAREJAMIENTO (MATCHING).....	12
3.2.6 - PREGUNTA ANIDADA (CLOZE).....	12
3.2.7 - ENSAYO Y TIPOS GENÉRICOS (ESSAY).....	13
3.3 - PATRONES DE DISEÑO DE SOFTWARE.....	13
3.3.1 - PATRÓN <i>FACTORY METHOD</i>	13
3.3.2 - PATRÓN <i>VISITOR</i>	14
3.4 - TECNOLOGÍAS DE SOPORTE	14
3.4.1 - XML Y EL MODELO DOM	14
3.4.2 – MATHJAX	15
3.4.3 - PANDOC.....	15
3.4.4 – LATEX Y CLASE exam	15
4 – TÉCNICAS Y HERRAMIENTAS.....	16
4.1 – METODOLOGÍA DE DESARROLLO	16
4.1.1 – DESARROLLO ITERATIVO E INCREMENTAL.....	16
4.1.2 – USO DE INTELIGENCIA ARTIFICIAL COMO APOYO COMPLEMENTARIO	16
4.1.3 – CONTROL DE VERSIONES CON GIT Y GITHUB	17
4.2 – LENGUAJE Y PLATAFORMA DE DESARROLLO	18



4.2.1 – JAVA 17	18
4.2.2 – JAVAFX 17	18
4.3 – HERRAMIENTAS DE PROCESAMIENTO DE DATOS	19
4.3.1 – JAXP (<i>JAVA API FOR XML PROCESSING</i>)	19
4.3.2 – EXPRESIONES REGULARES	19
4.3.3 – PANDOC	20
4.4 – GESTIÓN DEL PROYECTO APACHE MAVEN	20
4.5 – CALIDAD DEL CÓDIGO: SONARCLOUD	22
4.6 – ENTORNO DE DESARROLLO: ECLIPSE IDE	22
4.7 – DOCUMENTACIÓN Y REDACCIÓN DE LA MEMORIA: MICROSOFT WORD	23
4.8 – TESTING: JUNIT 5.....	23
5 – ASPECTOS RELEVANTES DEL DESARROLLO DEL PROYECTO.....	24
5.1 – ARQUITECTURA GENERAL DEL SISTEMA	24
5.2 – EL MODELO DE DATOS: LA JERARQUÍA DE QUESTION	25
5.2.1 – LA CLASE ABSTRACTA QUESTION	25
5.2.2 – LA IMPLEMENTACIÓN DE CADA SUBTIPO	25
5.3 – EL CICLO XML: PARSEO Y EXPORTACIÓN	26
5.3.1 – EL PARSEO DEL XML DE MOODLE	26
5.3.2 – QUESTIONFACTORY: EL PATRÓN <i>FACTORY METHOD</i> EN ACCIÓN	26
5.3.3 – LA EXPORTACIÓN A XML DE MOODLE	27
5.4 – EL PROCESAMIENTO DE PREGUNTAS CLOZE	27
5.4.1 – LA EXPRESIÓN REGULAR DE TOKENIZACIÓN	27
5.4.2 – EL ANÁLISIS DE ALTERNATIVAS	28
5.4.3 – LOS DOS MODOS DE VISUALIZACIÓN	28
5.5 – EL MOTOR DE EXPORTACIÓN A LATEX.....	29
5.5.1 – ESTRUCTURA DEL DOCUMENTO LATEX GENERADO	29
5.5.2 – LA CONVERSIÓN HTML A LATEX CON PANDOC	30
5.5.3 – EXTRACCIÓN DE IMÁGENES BASE64.....	30
5.6 – LA INTERFAZ DE USUARIO.....	31
5.6.1 – DISTRIBUCIÓN EN GESTORES ESPECIALIZADOS	31
5.6.2 – EL DIÁLOGO DE CREACIÓN DE PREGUNTAS	31
5.6.3 – LA CLASE HTMLCONSTANTS Y LA SEPARACIÓN DE ESTILOS	32
5.7 – EVALUACIÓN DE LA CALIDAD DEL CÓDIGO	33



6 – TRABAJOS RELACIONADOS	34
6.1 – GESTIÓN NATIVA MEDIANTE LA INTERFAZ WEB DE MOODLE	34
6.2 – EDITORES DE TEXTO PLANO Y PROCESADORES XML GENÉRICOS	35
6.3 – HERRAMIENTAS DE CONVERSIÓN DE TERCEROS Y PLUGINS	35
6.4 – POSICIONAMIENTO Y VALOR AÑADIDO DE LA SOLUCIÓN DESARROLLADA	35
7 – CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURAS	37
7.1 – CONCLUSIONES	37
7.2 – LÍNEAS DE TRABAJO FUTURAS	38
8 – BIBLIOGRAFÍA	39

ÍNDICE DE FIGURAS

Figura 4.1.3: Captura del repositorio de GitHub	17
Figura 4.2.2: Dependencias declaradas en el archivo pom.xml	19
Figura 4.4.1: Dependencia javafx-controls	20
Figura 4.4.2: Dependencia javafx-web	21
Figura 4.4.3: Dependencia junit-jupiter-api	21
Figura 4.4.4: Dependencia junit-jupiter-engine	21
Figura 5: Visualización general de la aplicación	24
Figura 5.4.3.1: Visualización de una pregunta Cloze en modo profesor	28
Figura 5.4.3.2: Visualización de una pregunta Cloze en modo alumno	29
Figura 5.6.2.1: Diálogo de creación de preguntas común a todos los tipos	32
Figura 5.6.6.2: Diálogo de creación de preguntas con los campos específicos de pregunta Verdadero/Falso	32
Figura 6.1: Visualización del banco de preguntas desde Moodle	34



Resumen

La gestión de bancos de preguntas masivos a través de la interfaz web de Moodle presenta notables limitaciones prácticas para el profesorado, especialmente en la reorganización y la exportación de contenido para exámenes impresos. Para dar respuesta a esta problemática, este Trabajo de Fin de Grado detalla el diseño y desarrollo de una aplicación de escritorio orientada a la gestión local y avanzada de estos recursos educativos. El sistema desarrollado actúa como editor bidireccional, visualizador interactivo y motor de exportación.

Implementada en Java 17 mediante la tecnología JavaFX, la herramienta parsea el formato XML nativo de Moodle apoyándose en patrones de diseño clásicos como el *Factory Method*. Además, integra un componente WebView con la librería MathJax que interpreta y renderiza fielmente el contenido HTML, las imágenes en Base64 y la sintaxis embebida de las preguntas Cloze tal y como se visualizarían en la plataforma virtual.

Finalmente, mediante la aplicación del patrón *Visitor* y la invocación de Pandoc como proceso externo, el sistema incorpora un motor de exportación capaz de transformar los bancos de preguntas en documentos compilables en LaTeX. Esto dota al docente de la capacidad de generar de forma automatizada exámenes físicos en blanco y solucionarios tipográficos de alta calidad listos para imprimir, cerrando la brecha técnica entre la evaluación digital y física.

Descriptores

Moodle, XML, Banco de preguntas, JavaFX, LaTeX, Patrones de diseño.



Abstract

Managing massive question banks through Moodle's web interface presents notable practical limitations for teachers, especially regarding the reorganization and exporting of content for printed exams. To address this problem, this Final Degree Project details the design and development of a desktop application aimed at the advanced, local management of these educational resources. The developed system acts as a bidirectional editor, an interactive visualizer, and an export engine.

Implemented in Java 17 using JavaFX technology, the tool parses Moodle's native XML format by relying on classic design patterns such as the Factory Method. Furthermore, it integrates a WebView component with the MathJax library that faithfully interprets and renders HTML content, Base64 images, and the embedded syntax of complex questions (Cloze) exactly as they would be displayed on the virtual platform.

Finally, through the application of the Visitor pattern and the invocation of Pandoc as an external process, the system incorporates an export engine capable of transforming question banks into compilable LaTeX documents. This provides the teacher with the ability to automatically generate blank physical exams and high-quality typographic answer keys ready for printing, closing the technical gap between digital and physical assessment.

Keywords

Moodle, XML, Question bank, JavaFX, LaTeX, Design patterns.



1 - INTRODUCCIÓN

La digitalización de la educación ha consolidado a las plataformas de gestión del aprendizaje como herramientas fundamentales en el entorno académico moderno. Entre ellas, Moodle destaca como la plataforma de código abierto más extendida a nivel mundial, con más de 400 millones de usuarios registrados en 237 países y más de 8.000 millones de preguntas de cuestionario almacenadas en sus servidores [1]. Su popularidad en el ámbito universitario se debe a su arquitectura modular, su capacidad de personalización y su amplio ecosistema de extensiones.

Uno de los pilares de la evaluación en Moodle son los bancos de preguntas, estructuras jerárquicas que permiten almacenar, clasificar y reutilizar preguntas de distintos tipos en múltiples cuestionarios. La riqueza tipológica que soporta Moodle constituye una de sus mayores fortalezas pedagógicas. Un banco de preguntas bien organizado permite al profesorado construir evaluaciones variadas y equilibradas de forma rápida y sencilla.

Sin embargo, la gestión de estos bancos a través de la interfaz web de Moodle presenta limitaciones prácticas notables cuando el volumen de preguntas crece. Reorganizar categorías, mover preguntas entre ellas o transformar el contenido a formatos físicos para exámenes impresos son operaciones que la plataforma no facilita de forma eficiente. Moodle ofrece la posibilidad de exportar e importar bancos de preguntas en un formato XML propio, pensado para la interoperabilidad entre instancias, pero que resulta complejo de editar manualmente y cuya estructura jerárquica no cuenta con soporte de edición visual fuera del entorno web.

Para dar respuesta a esta problemática, este TFG está pensado para diseñar y desarrollar un editor avanzado de bancos de preguntas Moodle. Se trata de una aplicación de escritorio construida en Java 17 mediante la tecnología JavaFX 17, orientada a la gestión local e independiente de estos recursos educativos. La aplicación actúa como editor, visualizador y exportador. Analiza sintácticamente el formato XML nativo de Moodle, transforma la información en un modelo de objetos interno jerárquico y ofrece una interfaz gráfica intuitiva para su manipulación estructurada.

La herramienta integra un componente WebView basado en WebKit con la librería matemática MathJax, lo que le permite interpretar y mostrar el contenido HTML de los enunciados, imágenes codificadas en Base64 y fórmulas matemáticas complejas tal y como las vería el estudiante en la plataforma virtual.

Además de garantizar la bidireccionalidad con la plataforma original mediante la exportación de vuelta a formato XML, la aplicación incorpora un motor de exportación a LaTeX que, apoyándose en la invocación de Pandoc [2] como proceso externo, convierte el HTML de los enunciados a sintaxis LaTeX limpia. El sistema extrae y almacena automáticamente las imágenes embebidas en Base64, generando documentos .tex compilables con la clase exam. Esto dota al profesorado de la capacidad de producir tanto exámenes en blanco como solucionarios con respuestas resaltadas, listos para imprimir.

Desde el punto de vista del diseño de software, el proyecto aplica de forma deliberada patrones clásicos del Gang of Four [3]. El patrón *Factory Method* gobierna la instanciación dinámica de los distintos tipos de pregunta durante el parseo XML, mientras que el patrón *Visitor* desacopla la lógica de exportación a LaTeX del modelo de datos, facilitando la extensión a nuevos formatos sin modificar las clases existentes.



Todo el código fuente y el histórico de desarrollo están disponibles en el repositorio de GitHub del proyecto:

<https://github.com/miguelito2003bcf/UBUTrabajoPreguntas>

El informe de calidad generado por SonarCloud puede visualizarse en:

https://sonarcloud.io/summary/overall?id=miguelito2003bcf_UBUTrabajoPreguntas&branch=main

Este documento constituye la memoria principal del proyecto. En ella se detallan los objetivos planteados, los conceptos teóricos necesarios para su comprensión, las técnicas y herramientas empleadas y los aspectos más relevantes del proceso de diseño e implementación. La documentación técnica complementaria se presenta en los anexos correspondientes.



2 - OBJETIVOS DEL PROYECTO

Este apartado detalla de forma precisa los objetivos que han guiado el diseño y el desarrollo de la aplicación. Se distinguen tres categorías: los objetivos funcionales, que definen las capacidades que el sistema debe ofrecer al usuario final; los objetivos técnicos, que recogen las metas de arquitectura, diseño e implementación y los objetivos de aprendizaje personal, que describen las competencias adquiridas durante el desarrollo.

2.1 - OBJETIVOS FUNCIONALES

Los objetivos funcionales definen las herramientas e interacciones que el sistema debe proporcionar al docente para gestionar su banco de preguntas de forma eficiente e independiente de la plataforma Moodle.

- **Gestión bidireccional de bancos de preguntas:** Importar archivos en formato XML nativo de Moodle extrayendo la jerarquía completa de categorías, preguntas de todos los tipos soportados y archivos multimedia incrustados en Base64. Exportar cualquier modificación a un XML compatible con Moodle, garantizando la integridad y la fidelidad de los datos originales.
- **Edición y reorganización interactiva:** Proveer una interfaz gráfica para añadir, editar, renombrar y eliminar categorías y preguntas. Soportar operaciones de *Drag & Drop* para mover preguntas entre categorías y reestructurar el árbol de categorías con soporte para fusión y movimiento como subcategoría.
- **Búsqueda y filtrado en tiempo real:** Localizar preguntas o categorías mediante barras de búsqueda con filtrado dinámico, así como filtros desplegables por tipo de pregunta, optimizando la navegación en bancos de datos de gran volumen.
- **Simulación de la vista del alumno:** Incorporar un visor que interprete HTML, muestre imágenes Base64 y renderice fórmulas matemáticas mediante MathJax. Para las preguntas Cloze, ofrecer una vista previa interactiva que replique el comportamiento real de los subproblemas embebidos tal y como los vería el alumno en Moodle.
- **Exportación a LaTeX:** Transformar el banco de preguntas completo en un documento .tex compilable con la clase exam, en dos modos: examen en blanco para alumnos y solucionario con respuestas resaltadas para el profesorado. El sistema gestiona automáticamente la extracción de imágenes Base64 a una carpeta de recursos adjunta.
- **Creación de nuevas preguntas:** Permitir al usuario crear nuevas preguntas de todos los tipos soportados mediante un diálogo de formularios dinámicos que adapta los campos de entrada según el tipo seleccionado. En un inicio no estaba pensado conseguir este objetivo, pero con el tiempo vimos que era factible, y tratamos de dejar la aplicación lo más completa posible.



- **Prevención de pérdida de datos:** Incorporar diálogos de confirmación antes de operaciones destructivas (eliminación o fusión de categorías, borrado de preguntas), garantizando la integridad de la información del usuario.

2.2 - OBJETIVOS TÉCNICOS

Los objetivos técnicos recogen las metas de implementación y diseño de software que rigen la arquitectura interna de la aplicación, asegurando su calidad, robustez y mantenibilidad.

- **Aplicación del patrón *Factory Method*:** Diseñar el análisis sintáctico del XML de modo que la instanciación de los distintos tipos de pregunta se delegue en la clase fábrica *QuestionFactory*, encapsulando la lógica de creación y desacoplando el *parser* de las clases concretas del modelo.
- **Aplicación del patrón *Visitor*:** Implementar la exportación a LaTeX de forma aislada. En lugar de que cada pregunta sepa cómo exportarse a sí misma, una clase externa como es *LaTeXExportVisitor* se encarga de esa tarea, manteniendo así las responsabilidades bien separadas.
- **Jerarquía de herencia para el modelo de preguntas:** Modelar los tipos de pregunta con la clase abstracta *Question* que centralice atributos compartidos, la infraestructura de renderizado HTML y el contrato del patrón *Visitor*.
- **Manipulación eficiente del DOM XML:** Utilizar las bibliotecas nativas de Java para recorrer, extraer y construir estructuras jerárquicas en memoria, garantizando fidelidad y compatibilidad con el formato de Moodle.
- **Procesamiento avanzado de sintaxis *Cloze*:** Desarrollar rutinas de *parsing* mediante expresiones regulares para tokenizar la sintaxis embebida, permitiendo tanto el resaltado de sintaxis para el profesor como el renderizado interactivo para el alumno.
- **Integración de Pandoc como proceso externo:** Invocar Pandoc como subprocesso mediante *ProcessBuilder* usando flujos de entrada/salida para convertir HTML a LaTeX sin ficheros temporales.
- **Separación de responsabilidades en la UI:** Distribuir la lógica de la interfaz en gestores especializados (*LayoutManager*, *TreeManager*, *TableManager*, *FileManager*) y centralizar los estilos CSS en *HtmlConstants*.
- **Gestión del proyecto con Maven:** Centralizar la gestión de dependencias y la construcción del proyecto mediante Apache Maven, asegurando la reproducibilidad del entorno de compilación en cualquier máquina.



2.3 - OBJETIVOS DE APRENDIZAJE PERSONAL

Estos objetivos describen las competencias y conocimientos adquiridos a nivel personal durante el desarrollo del proyecto.

- ***Aplicación práctica de la Ingeniería del Software:*** Llevar la teoría académica a la práctica desarrollando una aplicación completa desde cero, gestionando la complejidad de un proyecto de envergadura real desde el análisis de requisitos hasta la documentación final.
- ***Profundización en el ecosistema Java y JavaFX:*** Dominar el desarrollo de interfaces avanzadas con JavaFX, aprendiendo a gestionar el estado de la aplicación, el ciclo de vida de los componentes visuales y usar el componente WebView.
- ***Experiencia en transformación de formatos:*** Adquirir competencia práctica en la manipulación y transformación entre formatos de datos estructurados entendiendo los retos de la serialización y deserialización de datos complejos.
- ***Consolidación de patrones de diseño:*** Afianzar el conocimiento de patrones Gang of Four (GoF) a través de su implementación real, comprendiendo las implicaciones de cada decisión sobre la mantenibilidad del sistema.



3 - CONCEPTOS TEÓRICOS

Este capítulo expone los fundamentos teóricos necesarios para comprender el dominio del proyecto y las decisiones de diseño adoptadas. Se abordan la plataforma Moodle y su formato de intercambio XML, los tipos de pregunta gestionados por la aplicación, los patrones de diseño de software aplicados y las tecnologías de soporte empleadas.

3.1 - MOODLE Y LOS BANCOS DE PREGUNTAS

Moodle es un sistema de gestión del aprendizaje de código abierto publicado bajo licencia GPL. Desde su primera versión en 2002, se ha convertido en la plataforma LMS más utilizada mundialmente, con más de 400 millones de usuarios registrados en 237 países [1]. Su popularidad en el ámbito universitario se debe a su arquitectura modular, la capacidad de personalización y el amplio ecosistema de plugins disponible.

Dentro de Moodle, el banco de preguntas es el repositorio central donde los docentes almacenan y organizan las preguntas que utilizarán en sus cuestionarios. Estas preguntas se estructuran en una jerarquía de categorías y subcategorías, permitiendo organizar el material por tema, dificultad o cualquier otro criterio pedagógico. Una misma pregunta puede reutilizarse en múltiples evaluaciones, lo que hace del banco de preguntas un activo de gran valor para el profesorado.

Para facilitar la portabilidad entre distintas instancias, Moodle permite exportar e importar bancos de preguntas en varios formatos. El más completo es el formato XML de Moodle [9], que serializa la jerarquía completa de categorías y la totalidad de los datos de cada pregunta (enunciado en HTML, respuestas, fracciones de puntuación, retroalimentación, archivos adjuntos en Base64) en un único fichero estructurado. Este formato es el que la aplicación analiza, manipula y exporta.

3.2 - TIPOS DE PREGUNTA DE MOODLE

Moodle soporta una amplia variedad de tipos de pregunta, cada uno con una estructura de datos y una lógica de evaluación diferente. La documentación oficial de Moodle [7] describe más de una decena de tipos, mientras que en mi aplicación se cubren los siete más frecuentes en la práctica docente universitaria.

3.2.1 - OPCIÓN MÚLTIPLE (MULTICHOICE)

Es el tipo más extendido. Presenta al alumno un enunciado y una lista de opciones, de las cuales una o varias pueden ser correctas. Cada opción lleva una fracción de puntuación expresada como porcentaje: 100 para la respuesta completamente correcta, valores negativos para respuestas penalizadas y 0 para opciones neutras. El atributo booleano `single` determina si el alumno ve botones de radio (una sola respuesta posible) o casillas de verificación (varias respuestas posibles). El atributo `shuffleanswers` controla si las opciones se barajan en cada intento, una medida habitual para dificultar la copia entre alumnos.



3.2.2 - VERDADERO/FALSO (TRUEFALSE)

Presenta una afirmación y solicita al alumno evaluarla como verdadera o falsa. Se modela internamente como un caso especial de opción múltiple con exactamente dos respuestas. En el XML de Moodle las dos respuestas se serializan con los valores literales `true` y `false`, independientemente del idioma de la interfaz. La retroalimentación diferenciada para cada opción puede orientar al alumno que responde incorrectamente.

3.2.3 - RESPUESTA CORTA (SHORTANSWER)

Solicita al alumno introducir una respuesta textual libre. Moodle compara la respuesta con una lista de respuestas aceptadas, admitiendo comodines (* para cualquier secuencia de caracteres y ? para un carácter individual). El atributo `usecase` controla si la comparación distingue mayúsculas de minúsculas. Cada respuesta aceptada puede llevar una fracción de puntuación diferente, permitiendo así respuestas parcialmente correctas como es el caso de abreviaturas o sinónimos.

3.2.4 - NUMÉRICA (NUMERICAL)

Variante de la respuesta corta orientada a valores cuantitativos. El alumno introduce un número y Moodle lo compara con la respuesta correcta admitiendo un margen de error configurable (tolerancia). Esto resulta especialmente útil en preguntas de ciencias e ingeniería donde diferencias de redondeo o conversión de unidades no deben penalizarse. El margen de tolerancia se serializa junto con la respuesta en el XML.

3.2.5 - EMPAREJAMIENTO (MATCHING)

Presenta dos columnas de elementos y solicita al alumno establecer la correspondencia entre ellos. En el XML se serializa como una lista de pares, cada uno con un elemento de texto de enunciado y su respuesta correcta correspondiente. Moodle genera automáticamente los desplegables de selección para el alumno. Este tipo de pregunta resulta muy eficaz para evaluar vocabulario, relaciones conceptuales o asociaciones de definición.

3.2.6 - PREGUNTA ANIDADA (CLOZE)

Es el tipo más complejo de los soportados. El enunciado contiene tokens embebidos con una sintaxis propia de Moodle que definen subpreguntas inline [8]. Cada token sigue el patrón:

```
{puntos:TIP0:alternativa1~alternativa2~...}
```




El tipo puede ser multichoice y sus variantes de orientación, shortanswer, numerical y variantes multiresponse para selección múltiple en línea. Las alternativas pueden llevar fracciones prefijadas (%50%) o indicar la correcta con =. La retroalimentación de cada alternativa se indica tras un símbolo #. La aplicación tokeniza estos enunciados mediante expresiones regulares para renderizarlos de forma interactiva.

3.2.7 - ENSAYO Y TIPOS GENÉRICOS (ESSAY)

El tipo ensayo solicita una respuesta larga que requiere corrección manual por parte del docente. La aplicación lo gestiona como tipo genérico, mostrando el enunciado y un área de texto representativa sin respuesta correcta. Este mismo tratamiento se aplica a tipos menos frecuentes o no reconocidos, garantizando que ningún tipo de pregunta provoque un error de parseo aunque no esté plenamente soportado en la visualización.

3.3 - PATRONES DE DISEÑO DE SOFTWARE

Los patrones de diseño son soluciones reutilizables a problemas recurrentes en el diseño de software orientado a objetos. Fueron sistematizados por Gamma, Helm, Johnson y Vlissides [3] (conocidos colectivamente como el GoF) en su obra *Design Patterns: Elements of Reusable Object-Oriented Software* (1994, Addison-Wesley), obra de referencia fundamental de la ingeniería del software. Este proyecto aplica de forma deliberada dos de los 23 patrones descritos en dicha obra.

3.3.1 - PATRÓN *FACTORY METHOD*

El patrón *Factory Method* es un patrón de creación que define una interfaz para instanciar objetos, pero delega en métodos especializados la decisión sobre qué clase concreta instanciar. Su objetivo fundamental es desacoplar el código que utiliza un objeto del código que lo crea, de forma que añadir nuevas variantes no requiera modificar el código cliente.

El patrón resuelve el problema conocido como acoplamiento al tipo concreto: si el código del parser tuviese que saber qué clase instanciar para cada tipo de pregunta, cualquier adición de un nuevo tipo requeriría modificar el parser. Con la fábrica, el parser únicamente conoce la interfaz común (Question) y delega la responsabilidad de creación a QuestionFactory, que actúa como punto de extensión del sistema.



3.3.2 - PATRÓN *VISITOR*

El patrón *Visitor* es un patrón de comportamiento que permite añadir nuevas operaciones a una jerarquía de clases sin modificar sus elementos. Separa el algoritmo de la estructura de datos sobre la que opera, situando la lógica en un objeto visitante externo.

El patrón define dos participantes principales: la interfaz *Visitor*, que declara un método `visit()` sobrecargado para cada clase concreta de la jerarquía, y los elementos, que implementan un método `accept(visitor)` que invoca el `visit` correspondiente a su tipo. Este mecanismo se denomina *double dispatch*: la operación ejecutada depende tanto del tipo del visitante como del tipo del elemento, sin necesidad de casting ni comprobaciones de tipo en tiempo de ejecución.

Una de las ventajas más poderosas del patrón *Visitor* es el cumplimiento del principio Abierto/Cerrado: la jerarquía de clases está cerrada para modificación, pero abierta para extensión. Cada vez que se desea añadir una nueva operación, basta con añadir un nuevo visitante sin tocar las clases existentes.

3.4 - TECNOLOGÍAS DE SOPORTE

Además de los conceptos de dominio y diseño, el proyecto hace uso de varias tecnologías cuya comprensión resulta necesaria para entender las decisiones de implementación adoptadas.

3.4.1 - XML Y EL MODELO DOM

- XML (*eXtensible Markup Language*) [13] es un lenguaje de marcado de propósito general diseñado para almacenar y transportar datos estructurados de forma legible tanto por humanos como por máquinas. El formato XML de Moodle codifica la jerarquía de categorías y preguntas como un árbol de elementos anidados, con el nodo raíz `<quiz>` del que penden los nodos `<question>` tanto de tipo `category` (que marcan los cambios de contexto de categoría) como de los distintos tipos de pregunta.
- El DOM (*Document Object Model*) es una API estándar que representa un documento XML como un árbol de objetos en memoria. La elección de DOM frente a analizadores secuenciales basados en eventos (SAX) responde directamente a la naturaleza de editor interactivo de la aplicación. El usuario puede modificar cualquier nodo del árbol en cualquier momento, lo que requiere acceso aleatorio continuo que DOM proporciona de forma óptima. Los analizadores SAX, por el contrario, procesan el documento en un único paso sin mantener la representación en memoria, siendo más eficientes en memoria, pero inadecuados para la edición interactiva.



3.4.2 – MATHJAX

MathJax [4] es una biblioteca JavaScript de código abierto desarrollada y mantenida por el consorcio American Mathematical Society que renderiza notación matemática (LaTeX, MathML, AsciiMath) en el navegador sin instalación adicional. Se incluye mediante una etiqueta `<script>` que carga la librería desde una CDN (*Content Delivery Network*), lo que permite integrarla en cualquier página HTML sin modificar el entorno del cliente. MathJax interpreta las marcas matemáticas en tiempo de renderizado y produce gráficos vectoriales de alta calidad que escalan sin pérdida de resolución.

En este proyecto se incluye la configuración de MathJax en todos los documentos HTML generados dinámicamente por las clases del modelo, con soporte para los tres delimitadores habituales de LaTeX ($...$, $...$, $...$) para inline y $...$ para *display*). Esto garantiza que las fórmulas de los enunciados se rendericen correctamente en el componente WebView de JavaFX, replicando fielmente la experiencia visual de Moodle.

3.4.3 - PANDOC

Pandoc [2] es un conversor de documentos universal de código abierto creado por John MacFarlane, profesor de filosofía en la Universidad de California, Berkeley. Está implementado en Haskell y es capaz de transformar entre más de cuarenta formatos de marcado, incluyendo HTML, Markdown, LaTeX, DOCX, EPUB y muchos otros.

Su arquitectura interna se basa en un modelo de representación intermedia (*AST*, *Abstract Syntax Tree*) agnóstico del formato, con módulos de lectura (*readers*) y escritura (*writers*) independientes para cada formato. Esto garantiza que la conversión preserve la semántica del documento (énfasis, listas, tablas, imágenes, fórmulas matemáticas) aunque el formato de salida tenga limitaciones expresivas respecto al de entrada. En este proyecto se invoca como subprocesso del sistema operativo (ProcessBuilder de Java[6]) para convertir los fragmentos HTML de los enunciados a LaTeX limpio.

3.4.4 – LATEX Y CLASE exam

LaTeX es un sistema de composición tipográfica de alta calidad desarrollado por Leslie Lamport sobre el motor TeX de Donald Knuth. A diferencia de los procesadores de texto convencionales, en LaTeX el autor escribe el contenido acompañado de marcas de formato y un compilador calcula la composición final, optimizando la distribución del texto, los saltos de página y el espaciado entre elementos.

La clase exam es un paquete LaTeX diseñado específicamente para la elaboración de exámenes y cuestionarios con corrección automática o asistida. Proporciona entornos estructurales como questions para la lista de preguntas, choices y checkboxes para las opciones de respuesta, y solution para las soluciones del profesor. La opción de documento answers activa o desactiva globalmente la impresión de las soluciones, lo que permite generar el solucionario y el examen en blanco desde exactamente el mismo fichero .tex cambiando un único parámetro.



4 – TÉCNICAS Y HERRAMIENTAS

Este capítulo describe en detalle el ecosistema tecnológico, los lenguajes de programación, las bibliotecas y las herramientas de desarrollo que han hecho posible el diseño, la implementación y la validación de la aplicación. La selección de este conjunto de tecnologías no ha sido arbitraria, sino que responde a la necesidad de construir un sistema con máxima portabilidad multiplataforma, alta robustez en el procesamiento de datos en memoria y una experiencia de usuario fluida en entorno de escritorio. Cuando ha sido relevante, se justifica la elección frente a alternativas consideradas.

4.1 – METODOLOGÍA DE DESARROLLO

4.1.1 – DESARROLLO ITERATIVO E INCREMENTAL

El proyecto se ha gestionado siguiendo un enfoque de desarrollo iterativo e incremental, adaptado a las particularidades de un trabajo académico individual con orientación tutorial periódica. En lugar de planificar la totalidad de las funcionalidades desde el inicio y ejecutarlas en un único bloque, el desarrollo se organizó en iteraciones cortas de una a dos semanas. Al final de cada una de ellas, se disponía de un incremento funcional del sistema que podía ser revisado y validado.

En cada iteración, se seleccionaban las funcionalidades de mayor prioridad: primero el análisis sintáctico del XML y la visualización básica, después la edición interactiva y el *Drag & Drop*, posteriormente el renderizado Cloze y finalmente el motor de exportación LaTeX. Este enfoque permitió detectar y corregir problemas de diseño de forma temprana (especialmente en la jerarquía de clases y en la arquitectura del modelo de datos) antes de que se propagasen a otras partes del sistema.

A lo largo del proceso, el tutor del proyecto elaboró en paralelo una implementación propia del mismo trabajo. Esto favoreció una mejor comprensión de los desafíos técnicos específicos y una orientación fundamentada en cada revisión. Esta dinámica de trabajo conjunta facilitó la detección temprana de decisiones de diseño que, de haberse mantenido, habrían generado deuda técnica significativa en fases posteriores.

4.1.2 – USO DE INTELIGENCIA ARTIFICIAL COMO APOYO COMPLEMENTARIO

En determinados momentos puntuales del desarrollo, cuando el progreso se veía bloqueado por dudas técnicas concretas (como la construcción de la expresión regular para el tokenizador Cloze, la gestión de los flujos de entrada/salida en la invocación de Pandoc o el comportamiento de ciertos eventos de *Drag & Drop* en JavaFX), se recurrió al uso de herramientas de inteligencia artificial generativa como apoyo complementario para resolver esas dudas específicas.



Este uso puntual y focalizado de la IA, orientado a resolver bloqueos concretos de implementación, se alinea con las prácticas emergentes en el desarrollo de software profesional y académico, donde estas herramientas actúan como un recurso de consulta técnica adicional, similar al uso de la documentación oficial o de foros especializados, sin sustituir el razonamiento y las decisiones de diseño del desarrollador.

4.1.3 – CONTROL DE VERSIONES CON GIT Y GITHUB

Git es el sistema de control de versiones distribuido más utilizado en el desarrollo de software moderno. Permite registrar el historial completo de cambios del proyecto, trabajar en paralelo sobre distintas funcionalidades mediante ramas y revertir cambios en caso de error, proporcionando una red de seguridad esencial durante el desarrollo.

Además, los mensajes de confirmación de los cambios se han redactado siguiendo el estándar de *Conventional Commits* [16], garantizando un historial de versiones legible y estructurado.

El repositorio del proyecto se aloja en GitHub, donde se han ido subiendo las versiones estables del código a medida que se completaban los incrementos funcionales de cada iteración. GitHub ha actuado principalmente como repositorio remoto de respaldo y como registro del historial de versiones del proyecto, garantizando la preservación del código y permitiendo acceder a versiones anteriores en caso necesario.

El repositorio está disponible en:

<https://github.com/miguelito2003bcf/UBUTrabajoPreguntas>

En la *Figura 4.1.3* se puede observar el estado actual de mi repositorio.

 Malonsoalonso Se añade el archivo de anexos ✓	ee36570 · 1 minute ago	 44 Commits
 ArchivosXMLParaProbarLaAplicación	Remodelación del repositorio: limpieza de archivos que no va...	1 hour ago
 doc	Se añade el archivo de anexos	1 minute ago
 src	Se añaden las cabeceras de todas las clases de Java que mue...	2 hours ago
 .gitignore	Elimino la carpeta que no tenia ninguna función en nuestro r...	3 months ago
 LICENSE	Creación de la licencia MIT.	yesterday
 README.md	Se añaden los badges al README.md	5 days ago
 pom.xml	Últimas modificaciones para otorgar de color rojo a aquellas ...	last week

Figura 4.1.3: Captura del repositorio de GitHub



4.2 – LENGUAJE Y PLATAFORMA DE DESARROLLO

4.2.1 – JAVA 17

Java es un lenguaje de programación orientado a objetos, fuertemente tipado y compilado a bytecode que se ejecuta sobre la Máquina Virtual Java (JVM), lo que le proporciona portabilidad entre plataformas bajo el principio *Write Once, Run Anywhere*. Su maduro ecosistema de librerías, su robusto sistema de tipos y su amplio soporte de herramientas lo convierten en una elección sólida para el desarrollo de aplicaciones de escritorio de mediana y alta complejidad.

En este proyecto se utiliza Java 17, la versión LTS (*Long-Term Support*) publicada en septiembre de 2021, que garantiza soporte de seguridad y actualizaciones hasta 2029. Java 17 introduce mejoras relevantes para el proyecto como los bloques de texto para strings multilinea y las expresiones switch mejoradas. Aunque las características más determinantes son la madurez del lenguaje, la estabilidad de la JVM y la compatibilidad con JavaFX 17.

La elección de Java frente a otras alternativas se justifica por varios motivos: el dominio previo del lenguaje adquirido durante el grado en asignaturas de Programación Orientada a Objetos e Ingeniería del Software, la integración nativa con JavaFX 17, la disponibilidad de librerías estándar de procesamiento XML sin dependencias externas (`javax.xml`), la portabilidad entre sistemas operativos sin necesidad de recompilar y la idoneidad del lenguaje para la aplicación estricta de los principios de la POO y los patrones de diseño.

4.2.2 – JAVA FX 17

JavaFX 17 [5] es el framework moderno de desarrollo de interfaces gráficas de usuario para aplicaciones Java de escritorio. Fue inicialmente desarrollado por Sun Microsystems e integrado en el JDK por Oracle, siendo posteriormente liberado como proyecto de código abierto bajo el paraguas del proyecto OpenJFX. Con el JDK 11 se separó del JDK y se distribuye como módulo independiente, lo que requiere declararlo explícitamente como dependencia en Maven.

JavaFX ofrece frente a la biblioteca Swing (su predecesora) un motor de renderizado acelerado por hardware gráfico (GPU), un modelo de componentes basado en un grafo de escena (Scene Graph), soporte nativo para estilos CSS, enlace de datos reactivo mediante propiedades observables y un sistema de animaciones integrado. Para este proyecto resultan especialmente relevantes sus componentes: `TreeView` para la visualización jerárquica del árbol de categorías, `TableView` para el listado de preguntas con selección múltiple y ordenación, `SplitPane` para la distribución en paneles redimensionables por el usuario y `WebView` para el renderizado del HTML de enunciados.

El componente `WebView` es un navegador web embebido basado en el motor WebKit, el mismo motor que usan Safari y Chrome. Permite cargar y renderizar páginas HTML completas (con JavaScript, CSS y recursos embebidos) dentro de la interfaz JavaFX. Cada vez que el usuario selecciona una pregunta, la aplicación genera dinámicamente un documento HTML completo con la configuración de MathJax, los estilos CSS y el contenido de la pregunta y lo carga en el



WebView. El resultado es una fidelidad visual muy próxima a la que ofrece Moodle en el navegador. Cabe destacar que, si bien la estructura HTML y las imágenes codificadas en Base64 se procesan de forma estrictamente local sin necesidad de conexión a internet, la renderización de las fórmulas matemáticas sí requiere de una conexión activa en el momento de la visualización para poder cargar la librería MathJax desde su CDN.

La versión 17 se eligió por ser LTS, por ser la misma versión del JDK utilizado como compilador y por ser la versión disponible en el módulo `javafx-controls` y `javafx-web` declarados como dependencias en el proyecto tal y como se muestran a continuación en la *Figura 4.2.2*:

```
<javafx.version>17.0.6</javafx.version>

<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>${javafx.version}</version>
</dependency>
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-web</artifactId>
  <version>${javafx.version}</version>
</dependency>
```

Figura 4.2.2: Dependencias declaradas en el archivo `pom.xml`

4.3 – HERRAMIENTAS DE PROCESAMIENTO DE DATOS

4.3.1 – JAXP (JAVA API FOR XML PROCESSING)

La aplicación utiliza JAXP (Java API for XML Processing) [10], la API estándar incluida en el JDK de Java para el procesamiento de documentos XML, a través de su implementación del modelo DOM. JAXP proporciona las interfaces `DocumentBuilderFactory` y `DocumentBuilder` para construir el árbol DOM a partir de un fichero XML y `TransformerFactory` y `Transformer` para serializar el árbol DOM de vuelta a un fichero XML.

Se eligió la implementación DOM de JAXP frente a alternativas como SAX (analizador secuencial basado en eventos), StAX (analizador de cursores) o bibliotecas de terceros como JDOM, por dos razones principales. En primer lugar, al ser parte de la librería estándar del JDK 17 no introduce ninguna dependencia externa en el `pom.xml`. En segundo lugar, como ya se argumentó en el capítulo de conceptos teóricos, la naturaleza de editor interactivo de la aplicación requiere acceso aleatorio al árbol de nodos, algo que DOM proporciona directamente mientras que SAX y StAX no.

4.3.2 – EXPRESIONES REGULARES

El procesamiento de preguntas Cloze requiere un motor de expresiones regulares para identificar y extraer los tokens embebidos en los enunciados. Java proporciona soporte nativo



completo mediante el paquete `java.util.regex`, con las clases `Pattern` (compilación de la expresión) y `Matcher` (aplicación sobre cadenas de texto).

El patrón principal de la aplicación (`CLOZE_TOKEN_PAT`) se define como constante estática de la clase `ClozeQuestion` para evitar la recompilación en cada uso. Captura los siguientes grupos: el peso numérico opcional del token (`[0-9]*`), la constante del tipo de subpregunta mediante una alternancia explícita de más de doce variantes (`NUMERICAL|NM|MULTICHOICE_VS|MCVS|...`), y el bloque de alternativas mediante una cuantificación no codiciosa (`(.+?)`) hasta encontrar la llave de cierre no precedida por barra invertida (`((?<!\)\)\})`). El *flag* `dotall` permite que el punto (`.`) coincida también con saltos de línea, necesario para enunciados multilinea.

4.3.3 – PANDOC

Para la conversión de HTML a LaTeX en el proceso de exportación, la aplicación invoca Pandoc [2] como subprocesso externo mediante la clase `ProcessBuilder` de Java. El proceso recibe el fragmento HTML a través de la entrada estándar (`stdin`) y devuelve el resultado LaTeX por la salida estándar (`stdout`).

La comunicación se implementa con tres hilos concurrentes: el hilo principal que escribe en el `stdin` del proceso y el hilo que lee el `stdout` en paralelo, más un tercer hilo que consume el `stderr` para evitar bloqueos por desbordamiento del buffer del proceso hijo. Esta implementación correcta de la comunicación con subprocessos es crítica: si los *buffers* de `stdout` o `stderr` se llenan sin ser consumidos, el subprocesso se bloquea y la aplicación entra en interbloqueo.

Pandoc se eligió por gestionar correctamente los casos más complejos del HTML generado por los editores de Moodle. Implementar un parser HTML a LaTeX para esto habría supuesto un esfuerzo desproporcionado para los objetivos del proyecto y habría producido resultados menos robustos.

4.4 – GESTIÓN DEL PROYECTO APACHE MAVEN

Apache Maven [11] es la herramienta de automatización de construcción y gestión de dependencias más utilizada en el ecosistema Java. Permite declarar las dependencias del proyecto en un fichero descriptor `pom.xml` [15] (*Project Object Model*) y delega en Maven la descarga, el versionado y la resolución de conflictos entre librerías desde repositorios centrales como Maven Central.

El fichero `pom.xml` del proyecto declara las siguientes dependencias:

Para componentes visuales de JavaFX (`TreeView`, `TableView`, `SplitPane`, etc.)

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>${javafx.version}</version>
</dependency>
```

Figura 4.4.1: Dependencia `javafx-controls`

Para módulos WebView de JavaFX con el motor WebKit embebido.

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-web</artifactId>
  <version>${javafx.version}</version>
</dependency>
```

Figura 4.4.2: Dependencia javafx-web

Para la API de JUnit 5 para la escritura de tests unitarios.

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Figura 4.4.3: Dependencia junit-jupiter-api

Para el motor de ejecución de JUnit 5.

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Figura 4.4.4: Dependencia junit-jupiter-engine

El pom.xml configura además el maven-compiler-plugin:3.11.0 para compilar con Java 17 y el javafx-maven-plugin:0.0.8 de OpenJFX para facilitar la ejecución de la aplicación JavaFX desde Maven, indicando moodleviewer.Main como clase principal. Este plugin gestiona automáticamente el paso de los argumentos de módulos de JavaFX necesarios a la JVM, simplificando la ejecución en entornos de desarrollo.



4.5 – CALIDAD DEL CÓDIGO: SONARCLOUD

SonarCloud [12] es una plataforma de análisis estático de código basada en la nube que evalúa automáticamente la calidad y seguridad del software. Analiza el código fuente en busca de errores lógicos (*bugs*), vulnerabilidades de seguridad, código innecesariamente complejo (*code smells*) y duplicaciones, proporcionando métricas detalladas sobre la mantenibilidad del proyecto.

Tras cada análisis, SonarCloud emite un *Quality Gate*: un veredicto binario (*Passed / Failed*) que resume si el código cumple los umbrales de calidad definidos. Para este proyecto se obtuvo calificación A en las métricas de fiabilidad, mantenibilidad y seguridad, lo que indica que el código sigue estándares de buenas prácticas y carece de errores lógicos críticos.

4.6 – ENTORNO DE DESARROLLO: ECLIPSE IDE

Eclipse IDE [14] es uno de los entornos de desarrollo integrado más extendidos en el ecosistema Java, desarrollado por la Eclipse Foundation como proyecto de código abierto. Proporciona autocompletado inteligente de código, navegación avanzada entre clases y métodos, refactorización asistida, depuración integrada con puntos de interrupción y evaluación de expresiones y un potente motor de análisis estático en tiempo de edición que señala errores y avisos sin necesidad de compilar.

Para el desarrollo con JavaFX en Eclipse se utilizó el soporte de Maven integrado en el IDE, que permite importar proyectos Maven directamente, gestionar el ciclo de vida de Maven desde la interfaz gráfica y sincronizar automáticamente las dependencias del `pom.xml`. La integración con Git se realizó mediante el plugin EGit, nativo en las versiones recientes de Eclipse, que permite realizar commits, sincronizar con el repositorio remoto de GitHub y revisar el historial de cambios sin abandonar el entorno de desarrollo.

La elección de Eclipse frente a alternativas como IntelliJ IDEA o Visual Studio Code se fundamenta en la familiaridad previa adquirida durante el grado en las asignaturas de programación Java, donde Eclipse fue el IDE utilizado de forma sistemática. Esta familiaridad redujo significativamente la curva de aprendizaje del entorno de desarrollo, permitiendo centrarse en los aspectos arquitectónicos y de dominio del proyecto.



4.7 – DOCUMENTACIÓN Y REDACCIÓN DE LA MEMORIA: MICROSOFT WORD

La redacción, estructuración y maquetación de la presente memoria técnica se ha llevado a cabo utilizando el procesador de textos Microsoft Word. Aunque en el ámbito de la ingeniería del software es frecuente el empleo de sistemas de composición tipográfica basados en código como LaTeX, la planificación del proyecto y las estrictas restricciones del calendario en la fase final de entrega motivaron la elección de una herramienta en la que poseo más experiencia, y es más fácil e intuitiva.

Esta decisión responde a una estrategia de priorización de recursos. Dado el alto grado de complejidad técnica del código desarrollado (implementación de patrones de diseño, análisis léxico y motores de renderizado web), se decidió concentrar el esfuerzo y el tiempo disponible en la depuración del software y en las pruebas del sistema. Por ello, se descartó asumir los posibles tiempos de resolución de errores de compilación que exige LaTeX, optando en su lugar por un entorno más sencillo y visual.

Microsoft Word ha proporcionado un entorno de trabajo dinámico que ha permitido aplicar de forma visual e inmediata la normativa de estilos, márgenes y portadas exigida por la Universidad de Burgos. Además, su uso ha facilitado enormemente la inserción y redimensionado de las capturas de pantalla requeridas en esta memoria, así como la revisión ortográfica en tiempo real, agilizando el cierre y la entrega final del proyecto.

4.8 – TESTING: JUNIT 5

JUnit 5 es el framework de pruebas unitarias estándar en el ecosistema Java. La versión 5, publicada en 2017, introduce una arquitectura modular con separación entre la API de escritura de tests, el motor de ejecución y el lanzador. El proyecto declara en el `pom.xml` las dependencias previamente explicadas para su escritura y su ejecución, en ambos casos con `scope`, para que no se incluyan en el artefacto final de producción.

Las pruebas unitarias implementadas cubren la integridad del núcleo del modelo de datos y la capa de presentación visual. Específicamente, se ha validado la correcta instanciación y estructuración jerárquica de las categorías y subcategorías en memoria, así como la generación precisa del código HTML y metadatos para todos los tipos de preguntas soportados.



5 – ASPECTOS RELEVANTES DEL DESARROLLO DEL PROYECTO

Este capítulo expone en detalle las decisiones de diseño arquitectónico y las soluciones técnicas implementadas para resolver los desafíos más complejos del proyecto. Lejos de ser un mero inventario de componentes, se analizan los razonamientos detrás de cada decisión relevante, las alternativas consideradas y las consecuencias de las elecciones adoptadas sobre la mantenibilidad, la extensibilidad y el comportamiento del sistema.

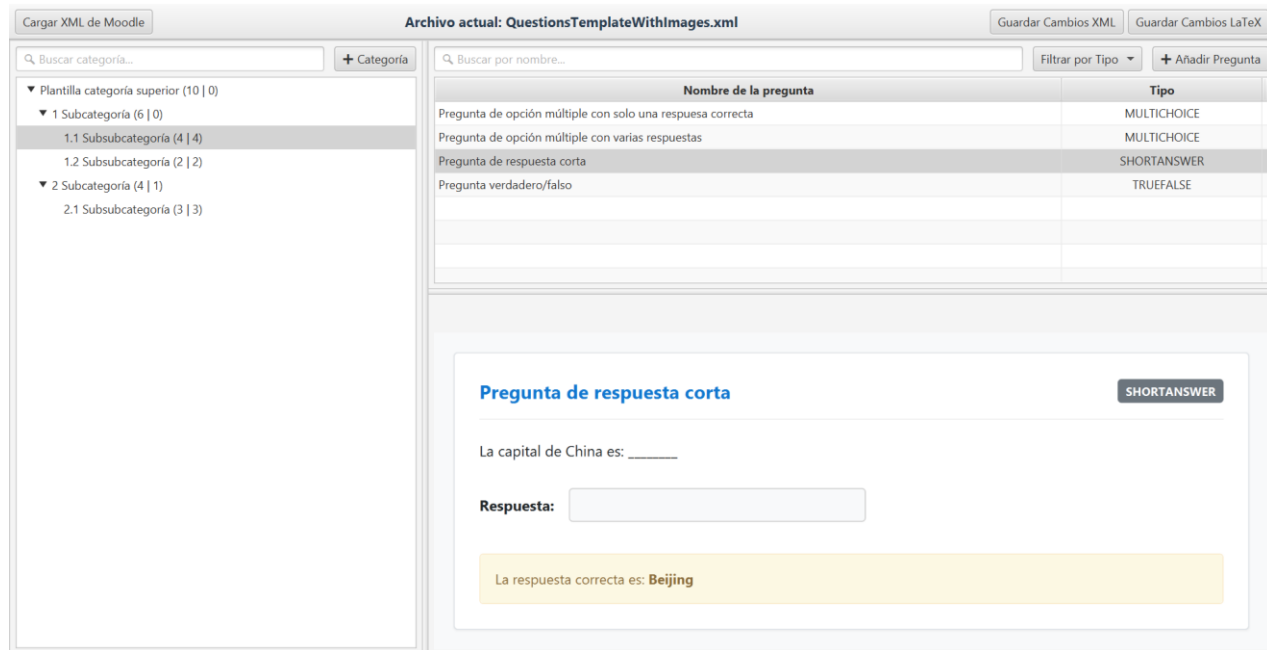


Figura 5: Visualización general de la aplicación

5.1 – ARQUITECTURA GENERAL DEL SISTEMA

La aplicación se estructura siguiendo un principio de separación de capas que distingue claramente tres niveles de responsabilidad: la capa de modelo, que contiene las clases de dominio en el paquete de modelo; la capa de parseo y exportación, con los componentes de transformación de datos en el paquete del parser y la capa de presentación, con los gestores de la interfaz JavaFX más la clase de utilidad `HtmlConstants` en el paquete de utilidad.

Esta organización garantiza que la lógica de dominio es completamente independiente de la interfaz gráfica y de los detalles del formato XML. Las clases del modelo no conocen la existencia de JavaFX, las clases de parseo no conocen los componentes visuales y los gestores de la UI no acceden directamente al XML. Cada cambio en una capa no obliga a modificar las demás.



5.2 – EL MODELO DE DATOS: LA JERARQUÍA DE QUESTION

5.2.1 – LA CLASE ABSTRACTA QUESTION

El diseño central del modelo de datos es la clase abstracta `Question`, de la que heredan las siete subclases concretas. Esta clase define los atributos comunes a todas las preguntas (`type`, `name`, `text`, `defaultGrade`, `penalty`, `generalFeedback`, `files`) junto con su infraestructura de renderizado HTML.

El método `getMoodleHeader()` genera la cabecera HTML compartida por todas las preguntas, donde encontramos el preámbulo del documento HTML con la configuración de MathJax cargada desde CDN, los estilos CSS del cuerpo de la página y de la tarjeta de contenido (definidos mediante las constantes de `HtmlConstants`), la cabecera con el nombre de la pregunta y el tipo en una insignia de color. Este método es responsable de garantizar que MathJax esté siempre disponible para cualquier tipo de pregunta que pueda contener notación matemática en su enunciado.

El método `getMoodleFooter()` cierra los elementos HTML abiertos en la cabecera y, si la pregunta tiene retroalimentación general, la renderiza en un bloque informativo de color azul antes del cierre del documento. La separación entre cabecera, contenido específico de cada tipo y pie permite que cada subclase solo necesite implementar la parte central sin preocuparse de la estructura del documento completo.

El método `processPluginFiles()` es otra pieza de infraestructura compartida. Recorre la lista de `MoodleFile` asociados a la pregunta y sustituye en el HTML de entrada cada referencia `@@PLUGINFILE@@/ruta/nombre.ext` por el URI de datos Base64 correspondiente. De este modo, las imágenes incrustadas en los enunciados se muestran correctamente en el `WebView` sin necesidad de acceso a ningún servidor.

5.2.2 – LA IMPLEMENTACIÓN DE CADA SUBTIPO

Cada subclase de `Question` encapsula los atributos específicos de su tipo y sobrescribe `getDetails()` para generar el HTML apropiado para su visualización:

- **MultichoiceQuestion:** Almacena `singleAnswer`, `shuffleAnswers` y la lista de `Answer`. Genera un desplegable con todas las opciones y sus porcentajes y una sección de respuestas correctas resaltadas para el profesor.
- **TrueFalseQuestion:** Almacena las dos `Answer` (verdadero y falso). Muestra dos `radio buttons` deshabilitados e indica la respuesta correcta en un bloque amarillo.
- **ShortAnswerQuestion:** Almacena `caseSensitive` y la lista de `Answer` aceptadas. Muestra un campo de texto y la respuesta con fracción 100 como respuesta correcta.
- **NumericalQuestion:** Almacena un único `Answer` y la tolerancia como cadena. Muestra el campo numérico y la respuesta correcta con el margen de error.
- **MatchingQuestion:** Almacena la lista de `MatchingPair`. Genera una tabla HTML con el enunciado de cada par a la izquierda y un desplegable con la respuesta correcta preseleccionada a la derecha.



- **ClozeQuestion:** Solo almacena el enunciado con la sintaxis embebida. La complejidad reside en los métodos de renderizado, detallados en el Apartado 5.4.
- **GenericQuestion:** Tipo por defecto para ensayo y tipos no reconocidos. Muestra el enunciado y un área de texto representativa.

5.3 – EL CICLO XML: PARSEO Y EXPORTACIÓN

5.3.1 – EL PARSEO DEL XML DE MOODLE

La clase XMLParser implementa el parseo en un único método público `parseMoodleXML(File)` que devuelve la categoría raíz del árbol construido. El proceso comienza construyendo el árbol DOM completo del fichero en memoria.

A continuación, obtiene la lista de todos los nodos `<question>` del documento (independientemente de su profundidad) y los recorre secuencialmente.

Los nodos de tipo `category` actualizan el puntero de categoría activa: se extrae la ruta completa, se limpia de prefijos de sistema (`$course$/top/`, `$module$/top/`) y se navega el árbol de categorías en memoria creando los nodos intermedios que no existan. El método privado `findOrCreateCategory()` implementa esta navegación de forma recursiva, dividiendo la ruta por el separador `/` y descendiendo nodo a nodo.

El resto de nodos delegan la creación del objeto `Question` en `QuestionFactory`. Antes de añadir la pregunta a la categoría activa, XMLParser extrae los atributos comunes y los asigna al objeto creado por la fábrica. Los ficheros adjuntos se construyen con los atributos `name`, `path`, `encoding` y el contenido Base64 del texto del nodo.

5.3.2 – QUESTIONFACTORY: EL PATRÓN *FACTORY METHOD* EN ACCIÓN

La clase `QuestionFactory` recibe el tipo de pregunta como cadena y el elemento DOM de la pregunta y devuelve la instancia polimórfica correcta. Implementa el método estático `createQuestion()` con una estructura `switch` sobre el tipo:

- **Para *matching*:** Recorre los nodos `<subquestion>` y construye la lista de `MatchingPair` extrayendo el texto del enunciado y el texto de la respuesta.
- **Para *multichoice*:** Extrae `single`, `shuffleanswers` y recorre los nodos `<answer>` construyendo la lista de `Answer` con fracción, texto y retroalimentación.
- **Para *truefalse*:** Construye las dos `Answer` detectando cuál tiene fracción 100 para asignarla a la respuesta verdadera.
- **Para *numerical*:** Extrae el primer nodo `<answer>` y el nodo `<tolerance>` como hijos directos.
- **Para *shortanswer*:** Extrae `usecase` y la lista de respuestas aceptadas.



- **Para cloze:** No necesita extracción adicional, la sintaxis está en el texto del enunciado.
- **Para cualquier tipo no reconocido:** Devuelve una `GenericQuestion`, garantizando que ningún tipo desconocido provoca un error.

La separación entre `XMLParser` (que extrae los atributos comunes) y `QuestionFactory` (que extrae los atributos específicos del tipo) respeta el principio de responsabilidad única y evita que la fábrica tenga que conocer el formato XML completo.

5.3.3 – LA EXPORTACIÓN A XML DE MOODLE

La clase `XMlexporter` realiza la operación inversa al *parser*. Crea un nuevo documento DOM vacío y lo rellena recorriendo recursivamente el árbol de categorías. El método privado `exportCategoryRecursive()` genera para cada categoría (excepto para la raíz) un nodo `<question type="category">` con la ruta acumulada y para cada pregunta invoca `writeQuestion()`.

El método `writeQuestion()` serializa los atributos comunes y mediante instancias de los operadores de tipo, los atributos específicos de cada subtipo. Un aspecto crítico es el uso correcto de secciones `CDATA`. El contenido HTML de los enunciados y las respuestas se envuelve en `doc.createCDATASection()` para preservar el marcado HTML sin escapar caracteres especiales como `<`, `>` y `&` que son válidos en HTML pero tienen significado especial en XML.

Para las preguntas de verdadero/falso, un detalle importante es que las respuestas se serializan con los valores literales `true` y `false` en lugar del texto localizado del idioma de la interfaz, garantizando la compatibilidad con Moodle independientemente del idioma configurado en la plataforma. Finalmente, el documento DOM se serializa a XML formateado mediante `TransformerFactory` con indentación de dos espacios y codificación UTF-8.

5.4 – EL PROCESAMIENTO DE PREGUNTAS CLOZE

El procesamiento de las preguntas Cloze constituye el mayor reto algorítmico del proyecto. Moodle almacena estas preguntas como texto plano con un sub-lenguaje embebido entre llaves, y la aplicación debe ser capaz tanto de mostrar esta sintaxis de forma legible al profesor como de simular visualmente el resultado interactivo que vería el alumno en la plataforma.

5.4.1 – LA EXPRESIÓN REGULAR DE TOKENIZACIÓN

El patrón `CLOZE_TOKEN_PAT` de la clase `ClozeQuestion` se define como constante estática compilada una sola vez. Identifica la estructura completa del token Cloze mediante grupos de captura. El grupo 1 captura el peso numérico opcional (`[0-9]*`), el grupo 2 captura la constante del tipo mediante una alternancia que cubre más de doce variantes y sus alias y el grupo 3 captura el bloque de alternativas con cuantificación no codiciosa (`(.+?)`) hasta la llave de cierre no escapada.

El lookbehind negativo (`(?<!\)\)\}`) en el delimitador final es crucial ya que permite que las alternativas de texto contengan llaves escapadas con barra invertida (como ocurre en las fórmulas LaTeX embebidas en los enunciados) sin que se interprete el cierre del token prematuramente. El



`flag Pattern.DOTALL` garantiza que el punto coincide también con saltos de línea, necesario para enunciados cuyas alternativas se distribuyen en varias líneas.

5.4.2 – EL ANÁLISIS DE ALTERNATIVAS

Una vez localizado el token, el método `parseClozeAlternatives()` descompone la cadena de alternativas usando el separador `~` y para cada alternativa, extrae la fracción de puntuación y el texto. La fracción puede expresarse de tres formas: el símbolo `=` al inicio indica fracción 1.0 (respuesta completamente correcta), el prefijo `%N%` indica fracción $N/100$ o la ausencia de prefijo indica fracción 0.0. El texto de la alternativa finaliza en el símbolo `#` si hay retroalimentación, o al final de la cadena si no la hay. Finalmente, se desescapan los caracteres especiales (`\`), `\~`, `\=`).

El resultado se encapsula en instancias de la clase interna `ClozeAlt`, cuyo único propósito es agrupar la fracción de puntuación (`double`) y el texto (`String`) de una alternativa. Esta clase interna privada sigue el principio de mínimo alcance donde su uso está restringido al contexto donde tiene sentido.

5.4.3 – LOS DOS MODOS DE VISUALIZACIÓN

La clase `ClozeQuestion` ofrece dos modos de visualización controlados por el flag estático `MODO_PREVIA_ALUMNO`, accesible desde la interfaz mediante un botón `toggle`:

- **Modo profesor (sintaxis Cloze):** El método `highlightClozeSyntax()` recorre el texto con el `Matcher` del patrón y envuelve cada token localizado con un elemento `<span>` con fondo gris y texto en azul monoespacio según el estilo definido en la clase de utilidad. El texto del token se escapa para HTML antes de insertarse, garantizando que los caracteres especiales del XML se muestren correctamente como texto y no sean interpretados por el parser HTML del `WebView`.

Así es como se vería desde nuestra aplicación, remarcando la sintaxis Cloze según la *Figura 5.4.3.1*.

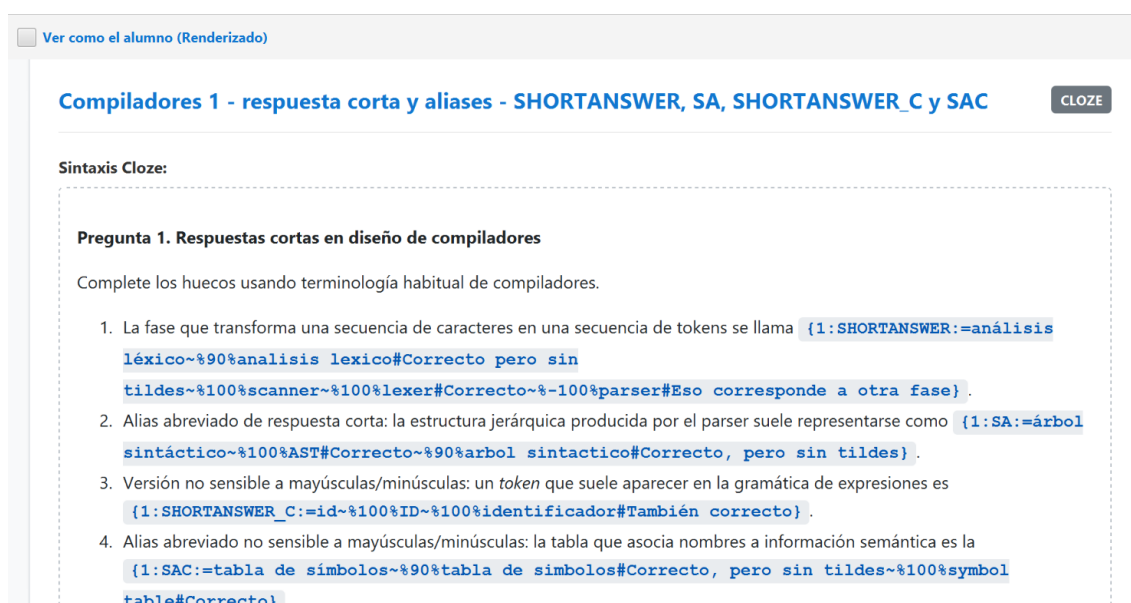


Figura 5.4.3.1: Visualización de una pregunta Cloze en modo profesor

- **Modo alumno (vista previa interactiva):** El método `renderCloze()` sustituye cada token por el widget HTML apropiado mediante el método `buildClozewidget()`. Según el tipo del subproblema genera un `<select>` con las opciones para los tipos MULTICHOICE planos, con la respuesta correcta marcada y los porcentajes visibles; campos `<input type='text'>` con la respuesta correcta como placeholder para SHORTANSWER y NUMERICAL o grupos de radio buttons y checkboxes para los tipos verticales y horizontales. Todos los widgets se generan como disabled para dejar claro que es una vista previa no editable.

Así es como se vería desde nuestra aplicación, sustituyendo el fragmento textual de Cloze por su vista previa como se puede observar en la *Figura 5.4.3.2*:



Ver como el alumno (Renderizado)

Compiladores 1 - respuesta corta y alias - SHORTANSWER, SA, SHORTANSWER_C y SAC CLOZE

Vista previa del alumno:

Pregunta 1. Respuestas cortas en diseño de compiladores

Complete los huecos usando terminología habitual de compiladores.

1. La fase que transforma una secuencia de caracteres en una secuencia de tokens se llama .
2. Alias abreviado de respuesta corta: la estructura jerárquica producida por el parser suele representarse como .
3. Versión no sensible a mayúsculas/minúsculas: un *token* que suele aparecer en la gramática de expresiones es .
4. Alias abreviado no sensible a mayúsculas/minúsculas: la tabla que asocia nombres a información semántica es la .
5. Varias respuestas correctas y retroalimentación: una estrategia descendente sin retroceso usada en compiladores docentes es .

Figura 5.4.3.2: Visualización de una pregunta Cloze en modo alumno

5.5 – EL MOTOR DE EXPORTACIÓN A LATEX

5.5.1 – ESTRUCTURA DEL DOCUMENTO LATEX GENERADO

La clase `LaTeXExporter` orquesta la generación del fichero `.tex` completo. El preámbulo incluye la clase `exam` (con o sin la opción `answers` según el modo seleccionado por el usuario), los paquetes estándar (`inputenc`, `babel`, `amsmath`, `graphicx`, `geometry`, etc.) y la definición de los entornos personalizados que mapean los tipos de pregunta de Moodle a la semántica de la clase `exam`:



- ***unaRespuesta* / *unaRespuestaEnLinea***: Envuelven `\begin{checkboxes}` para preguntas multichoice de una sola respuesta, en orientación vertical y horizontal respectivamente.
- ***variasRespuestas* / *variasRespuestasEnLinea***: Ídem con `\checkboxchar{\Large$\Box$}` para preguntas de selección múltiple.
- ***emparejar***: Entorno para los pares de emparejamiento, con checkboxes grandes en azul para destacar visualmente la columna de respuesta.

El documento LaTeX también define el comando `\cgoCorrectChoice`, que en modo solucionario muestra la respuesta correcta enmarcada en un recuadro azul, y en modo examen la muestra con el recuadro en blanco invisible, sin cambiar la disposición del documento.

5.5.2 – LA CONVERSIÓN HTML A LATEX CON PANDOC

El método `htmlToLatex()` del visitante construye un `ProcessBuilder` con los argumentos `--from html --to latex --wrap=none`. La opción `--wrap=none` desactiva el ajuste automático de línea de Pandoc, que de otra forma insertaría saltos de línea en el LaTeX generado que podrían romper comandos distribuidos en varias líneas.

Pandoc recibe el HTML por `stdin` y devuelve el LaTeX por `stdout`. Para evitar el interbloqueo descrito en el apartado 4.3.3, los flujos se gestionan con hilos separados: un hilo lanza y cierra el escritor del `stdin`, otro hilo lee completamente el `stdout` en un `StringBuilder` y un tercer hilo consume el `stderr`. El hilo principal espera hasta que el proceso termina antes de leer el resultado.

5.5.3 – EXTRACCIÓN DE IMÁGENES BASE64

Cuando el HTML de un enunciado contiene imágenes embebidas como URIs de datos Base64, la aplicación las extrae y guarda como ficheros en la carpeta `_imagenes` adjunta al documento LaTeX. El visitante detecta el patrón `data:image/TYPE;base64,DATOS` en el HTML, decodifica el contenido mediante `java.util.Base64.getDecoder().decode()`, escribe el fichero de imagen con un nombre generado secuencialmente y sustituye la referencia URI inline por una instrucción `\includegraphics{./NOMBRE_IMAGENES_DIR/imagenN.ext}` de LaTeX. Esta solución garantiza que el documento `.tex` sea compilable en cualquier equipo que tenga la carpeta de imágenes en la ubicación relativa esperada.



5.6 – LA INTERFAZ DE USUARIO

5.6.1 – DISTRIBUCIÓN EN GESTORES ESPECIALIZADOS

La interfaz de usuario está organizada siguiendo un principio de delegación por responsabilidad, en la que la clase `Main` actúa como orquestador que inicializa todos los componentes y los conecta mediante listeners de eventos, pero el control fino de cada área se delega en gestores estáticos especializados.

`LayoutManager` construye y ensambla la escena principal. Organiza los componentes en una `ToolBar` superior con los botones de apertura, guardado y exportación y una etiqueta con el nombre del fichero activo. Debajo sitúa un `SplitPane` horizontal con una proporción de 30/70, que separa el árbol de categorías del área de trabajo. El área de trabajo contiene un segundo `SplitPane` vertical con una proporción de 50/50 que separa la tabla de preguntas del visor de las mismas.

`TreeManager` configura el `TreeView` de categorías con celdas personalizadas mediante la clase interna `CategoryTreeCell`. Cada celda muestra el nombre de la categoría junto con dos contadores que son el total de preguntas en la rama completa (incluyendo subcategorías) y el número de preguntas directas de esa categoría. El gestor configura los eventos de *Drag & Drop* para mover categorías completas, con validación de destinos (el sistema impide que una categoría se mueva a sí misma o a uno de sus descendientes) y ofrece al usuario la opción de mover como subcategoría o fusionar preguntas. El atajo de clic con `Ctrl` colapsa o expande recursivamente toda la rama.

`TableManager` configura el `TableView` de preguntas con dos columnas (nombre y tipo) cuyas anchuras se vinculan proporcionalmente al ancho de la tabla. Habilita la selección múltiple para permitir mover o eliminar varias preguntas a la vez y configura el *Drag & Drop* de las filas seleccionadas hacia el árbol de categorías.

`FileManager` encapsula toda la lógica de diálogos de fichero. El método `openXML()` lanza el `FileChooser` con filtro de extensión `.xml`, delega el parseo en `XMLParser` y devuelve el resultado envuelto en `Optional`. El método `saveXML()` guarda el árbol actual mediante `XMLExporter`. El método `exportLaTeX()` presenta primero un diálogo de confirmación para elegir el modo (solucionario o examen en blanco) y después el diálogo de guardado.

5.6.2 – EL DIÁLOGO DE CREACIÓN DE PREGUNTAS

La clase `AddQuestionDialog` extiende `Dialog<Question>` de `JavaFX` para integrarse con el flujo de diálogos modales de la aplicación. Presenta un menú desplegable para seleccionar el tipo de pregunta y un panel dinámico que actualiza sus campos en tiempo de ejecución según el tipo seleccionado.

El panel dinámico se implementa con un `VBox` cuyo contenido se reconstruye mediante un `ChangeListener` en el `ComboBox` del tipo. Cuando el usuario cambia el tipo, el listener limpia el `VBox` y añade los controles específicos del nuevo tipo: para `multichoice`, un área desplazable con filas editables de texto de respuesta y fracción; para `matching`, pares de campos para enunciado y respuesta; para `numerical`, un campo de tolerancia adicional; para `cloze`, un área de texto grande para editar la sintaxis embebida. El constructor del resultado en el método `setResultConverter` delega en `QuestionFactory` para crear la instancia apropiada a partir de los valores introducidos.



En la *Figura 5.6.2.1* se muestran los campos que habría que rellenar de forma común a todos los tipos de pregunta:

Figura 5.6.2.1: Diálogo de creación de preguntas común a todos los tipos

Categoría: 1.1 Subsubcategoría

Tipo de pregunta	Verdadero / Falso ▾
Nombre de la pregunta	
Enunciado de la pregunta	
Puntuación por defecto	1
Retroalimentación general	
Penalización por intento	0.3333333

Y para el caso concreto por ejemplo de una pregunta de Verdadero/Falso los campos que aparecerían serían los siguientes como se puede observar en la *Figura 5.6.2.2*:

Respuesta correcta	Verdadero ▾
Retroalimentación para la respuesta 'Verdadero'	
Retroalimentación para la respuesta 'Falso'	

Figura 5.6.6.2: Diálogo de creación de preguntas con los campos específicos de pregunta Verdadero/Falso

5.6.3 – LA CLASE HTMLCONSTANTS Y LA SEPARACIÓN DE ESTILOS

Toda la presentación visual del panel de detalles (colores de fondo, márgenes, tipografías, estilos de cuadros de retroalimentación, estilos de inputs de formulario) se centraliza en la clase final no instanciable `HtmlConstants`. Esta clase define 14 constantes estáticas de cadenas de estilo CSS inline, agrupadas por función:

- **Estructura principal:** `HTML_BODY`, `HTML_CARD`, `HEADER_FLEX`
- **Tipografía y etiquetas:** `TITLE`, `BADGE`, `TEXT_BASE`, `LABEL_BOLD`
- **Feedback y alertas:** `FEEDBACK_GENERAL` (fondo azul claro), `FEEDBACK_WARNING` (fondo amarillo claro)



- **Formularios y disposición:** FLEX_ROW, FLEX_ROW_START, INPUT_BASE, TABLE_LAYOUT
- **Específicos Cloze:** CLOZE_CODE_BLOCK, CLOZE_RENDER_BLOCK, CLOZE_HIGHLIGHT

Esta clase es el único punto donde modificar cualquier aspecto del estilo visual del visor, sin tocar la lógica de ninguna clase de pregunta.

5.7 – EVALUACIÓN DE LA CALIDAD DEL CÓDIGO

El análisis estático del código con SonarCloud se ha ejecutado sobre el repositorio GitHub del proyecto. Los resultados del *Quality Gate* personalizado muestran calificación A en las tres métricas principales evaluadas: fiabilidad (ausencia de bugs críticos en la lógica del programa), mantenibilidad (nivel de deuda técnica y code smells) y seguridad (ausencia de vulnerabilidades y hotspots).

Entre los aspectos valorados positivamente por SonarCloud destaca la consistencia del tratamiento de excepciones, la ausencia de recursos no cerrados y la correcta visibilidad de los miembros de las clases.



6 – TRABAJOS RELACIONADOS

En este capítulo se realiza un análisis crítico de las soluciones existentes para la gestión de bancos de preguntas de Moodle y la generación de exámenes, evaluando sus ventajas, sus limitaciones y justificando la necesidad de la solución propuesta.

6.1 – GESTIÓN NATIVA MEDIANTE LA INTERFAZ WEB DE MOODLE

La primera y más evidente alternativa es la propia interfaz web nativa de Moodle. Como sistema integrado en la plataforma, cualquier cambio se refleja inmediatamente y queda disponible para las evaluaciones. El editor Atto de Moodle ofrece una interfaz familiar para la redacción de enunciados. Su principal fortaleza es la inmediatez ya que no requiere exportar ni importar ficheros y el profesorado ya está familiarizado con la interfaz.

Sin embargo, la gestión nativa presenta graves deficiencias cuando se trabaja con volúmenes masivos de preguntas. Al ser una aplicación web renderizada del lado del servidor, operaciones como mover decenas de preguntas entre categorías requieren múltiples recargas de página y navegación por menús poco intuitivos. La plataforma carece de mecánicas ágiles como el *Drag & Drop* para reorganización masiva. La dependencia de conexión a internet impide el trabajo offline y aumenta los tiempos de respuesta.

Finalmente, Moodle no ofrece vía directa para exportar un examen a formato tipográfico de alta calidad, obligando al profesorado a depender de la impresión desde el navegador, que frecuentemente descoloca imágenes y rompe el formato de tablas.

Así se vería Moodle como en la *Figura 6.1*, en este caso el banco de preguntas:

The screenshot shows the Moodle question bank interface. At the top, there are navigation links: MoodleLocal, Página Principal, Área personal, Mis cursos, and Administración del sitio. On the right, there are user icons and a 'Modo de edición' toggle. The main heading is 'Banco de preguntas'. Below it, there are filter options: 'Coincidir' (Todos), 'de los siguientes:', 'Coincidir' (Categoría), 'Escriba o seleccione...', 'Categoría 1 (4)', and 'Mostrar también preguntas de las subcategorías'. There are also buttons for 'Mostrar', 'Limpiar filtros', and 'Aplicar filtros'. Below the filters, there is a table of questions. The table has columns: 'Pregunta', 'Acciones', 'Estado', 'Versión', 'Creado por', 'Comentarios', '¿Necesita revisión?', 'Índice de facilidad', 'Eficiencia discriminativa', 'Uso', 'Último uso', and 'Modificada por'. The first two rows of the table are visible, showing questions about 'Pregunta de emparejamiento' and 'Pregunta clothe'.

Pregunta	Acciones	Estado	Versión	Creado por	Comentarios	¿Necesita revisión?	Índice de facilidad	Eficiencia discriminativa	Uso	Último uso	Modificada por
Pregunta de emparejamiento	Editar	Listo	v1	Miguel Alonso 18 de febrero de 2026, 18:03	0	-	No disponible	No disponible	0	Nunca	Miguel Alonso 18 de febrero de 2026, 18:03
Pregunta clothe	Editar	Listo	v1	Miguel Alonso 19 de febrero de 2026, 13:16	0	-	No disponible	No disponible	0	Nunca	Miguel Alonso 19 de febrero de 2026, 13:16

Figura 6.1: Visualización del banco de preguntas desde Moodle



6.2 – EDITORES DE TEXTO PLANO Y PROCESADORES XML GENÉRICOS

Una práctica habitual entre usuarios avanzados consiste en descargar el XML de Moodle y modificarlo con editores de código como Notepad++, Sublime Text o Visual Studio Code. Esta aproximación resuelve el problema de la latencia web y permite operaciones de búsqueda y reemplazo masivas de forma completamente offline.

No obstante, la manipulación manual del XML entraña un alto riesgo de error humano. Un simple error sintáctico (olvidar cerrar una etiqueta CDATA, un corchete en una pregunta Cloze o un atributo mal formado) corrompe el fichero completo y Moodle rechaza la importación, frecuentemente sin mensajes de error descriptivos sobre la ubicación del problema.

Además, el docente no puede previsualizar cómo se renderizarán las fórmulas matemáticas ni verificar si el HTML incrustado es correcto, convirtiendo la edición de preguntas complejas en un proceso opaco y propenso a errores lógicos invisibles.

6.3 – HERRAMIENTAS DE CONVERSIÓN DE TERCEROS Y PLUGINS

En el ecosistema de la comunidad docente existen scripts y herramientas aisladas: macros de Microsoft Word para exportar preguntas a XML, plugins instalables en el servidor Moodle para generar documentos PDF o conversores online para tipos específicos. Estas herramientas automatizan partes del flujo de trabajo y reducen la curva de aprendizaje para tipos básicos de preguntas.

Sin embargo, suelen sufrir de severa falta de mantenimiento, quedando obsoletas e incompatibles con las nuevas versiones del formato XML de Moodle. La mayoría son unidireccionales y no ofrecen soporte integral para la diversidad tipológica del sistema, fallando habitualmente con preguntas Cloze complejas, imágenes en Base64 o notación matemática. Al depender de entornos de terceros, tampoco garantizan la privacidad de los contenidos del banco de preguntas.

6.4 – POSICIONAMIENTO Y VALOR AÑADIDO DE LA SOLUCIÓN DESARROLLADA

Tras el análisis anterior, se evidencia un vacío tecnológico, en donde se percibe la ausencia de una herramienta integral que unifique la agilidad de una aplicación de escritorio local con la fidelidad visual de un entorno web, añadiendo la capacidad de generar documentos físicos de calidad editorial. La aplicación desarrollada supera las limitaciones descritas mediante cuatro contribuciones principales:

- **Agilidad frente al entorno web:** Los gestores interactivos de JavaFX eliminan la fricción de la paginación web, permitiendo reestructurar bancos completos mediante *Drag & Drop* de forma instantánea y offline.



- ***Seguridad estructural frente a la edición manual:*** El parseo DOM con la fábrica de preguntas abstrae al usuario de la complejidad sintáctica del XML. Cualquier modificación genera un archivo validado y compatible, eliminando el riesgo de corrupción.
- ***Fidelidad visual integrada:*** La integración del WebView y MathJax replica la experiencia visual de Moodle. El profesor puede revisar el comportamiento de ejercicios Cloze en tiempo real validando su apariencia antes de la exportación.
- ***Puente tecnológico hacia la tipografía física:*** El motor LaTeXExportVisitor, con las rutinas de saneamiento de datos y extracción de imágenes, cierra el ciclo educativo generando exámenes impresos de formato profesional con un solo clic.



7 – CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURAS

7.1 – CONCLUSIONES

La realización de este TFG ha culminado con el desarrollo exitoso de un editor avanzado de bancos de preguntas Moodle, con una aplicación de escritorio plenamente funcional que resuelve una carencia técnica real en el ecosistema de la docencia. Tras someter la herramienta a pruebas de validación con bancos de preguntas complejos, se extraen las siguientes conclusiones:

- **Consecución de los objetivos funcionales:** Se ha implementado un flujo de trabajo bidireccional estable. El sistema importa la jerarquía completa de un XML, permite su reestructuración mediante operaciones interactivas intuitivas (*Drag & Drop*, menús contextuales, búsqueda en tiempo real) y exporta los resultados con compatibilidad estricta con Moodle. El visor, gracias al uso de WebView y MathJax, permite previsualizar el comportamiento de preguntas de varios tipos con fidelidad gráfica total.
- **Robustez arquitectónica y escalabilidad:** La aplicación estricta de la POO y la delegación de responsabilidades mediante los patrones *Factory Method* y *Visitor* han demostrado ser decisiones sumamente eficaces. Este desacoplamiento garantiza que la incorporación de nuevos tipos de pregunta en el futuro no requiera reescribir el núcleo lógico del programa.
- **Resolución de la brecha digital-física:** La integración de Pandoc y los algoritmos de saneamiento permiten generar documentos .tex limpios y compilables, otorgando al docente la capacidad de producir exámenes físicos y solucionarios de forma desatendida.
- **Importancia de la orientación técnica:** La dinámica de desarrollo con orientación tutorial, en la que el tutor elaboró en paralelo un proyecto similar, demostró ser una estrategia formativa de gran valor. Esta proximidad técnica entre tutor y alumno facilitó una retroalimentación fundada en la experiencia directa del mismo problema y no solo en criterios teóricos.
- **Aprendizaje del ciclo de vida completo:** El desarrollo íntegro del proyecto desde el análisis de requisitos hasta la documentación final ha supuesto una consolidación profunda en Java 17, JavaFX, diseño de interfaces con gestores especializados, manipulación de DOM XML, comunicación con subprocesos del sistema operativo y análisis estático de calidad de código.

En definitiva, el proyecto ha demostrado que es posible construir una herramienta de productividad docente de calidad profesional aplicando los principios de la Ingeniería del Software aprendidos durante el grado, combinando tecnologías heterogéneas (Java 17, JavaFX 17, XML/DOM, expresiones regulares, Pandoc y LaTeX) de forma coherente y mantenible.



7.2 – LÍNEAS DE TRABAJO FUTURAS

La arquitectura modular de la aplicación invita a plantear diversas mejoras y expansiones en iteraciones futuras:

- **Compilación directa a PDF:** Integrar la invocación de compiladores LaTeX directamente desde el código Java, devolviendo al usuario un PDF finalizado con un solo clic y ocultando por completo la complejidad de LaTeX al usuario final.
- **Sincronización mediante la API REST de Moodle:** Desarrollar un módulo de red que se autentifique contra la API REST de Moodle, descargue el banco directamente desde el servidor universitario y sincronice los cambios de vuelta, eliminando el trasiego manual de ficheros XML.
- **Módulo de estadísticas del banco:** Implementar un panel de análisis estadístico del banco de preguntas con métricas visuales sobre la distribución de calificaciones, el balance de tipos por categoría y el índice de dificultad medio, ayudando al profesorado en la planificación de evaluaciones equilibradas.
- **Internacionalización:** Extraer todas las cadenas de texto estáticas de la interfaz a archivos de propiedades para facilitar la traducción a otros idiomas y ampliar el alcance de la herramienta a comunidades docentes de habla no hispana.
- **Ampliación de la cobertura de pruebas:** Desarrollar una batería de pruebas unitarias e integradas más exhaustiva, especialmente para el motor de tokenización Cloze, el exportador LaTeX y el ciclo completo de parseo y exportación XML.



8 – BIBLIOGRAFÍA

- [1] LMS Daily. *Moodle LMS now officially powers 400 million users around the world*. LMS Daily, noviembre 2023. Disponible en: <https://lmsdaily.com/moodle-lms-now-officially-powers-400-million-users-around-the-world/> [Internet; Accedido: 31-05-2026]
- [2] J. MacFarlane. *Pandoc: a universal document converter*. Universidad de California Berkeley, 2006–2025. Disponible en: <https://pandoc.org/> [Internet; Accedido: 31-05-2026]
- [3] E. Gamma, R. Helm, R. Johnson y J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 0-201-63361-2.
- [4] MathJax Consortium. *MathJax: Beautiful and accessible math in all browsers*. Disponible en: <https://www.mathjax.org/> [Internet; Accedido: 31-05-2026]
- [5] Oracle Corporation / OpenJFX. *JavaFX: Getting Started with JavaFX (Release 8 / JDK 17)*. Oracle Documentation, 2024. Disponible en: <https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm> [Internet; Accedido: 31-05-2026]
- [6] Oracle Corporation. *Java SE 17 API Specification*. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/> [Internet; Accedido: 31-05-2026]
- [7] Moodle HQ. *Moodle Documentation: Question types*. Moodle Docs, 2024. Disponible en: https://docs.moodle.org/en/Question_types [Internet; Accedido: 31-05-2026]
- [8] Moodle HQ. *Moodle Documentation: Embedded answers (Cloze) question type*. Moodle Docs, 2024.
Disponible en: [https://docs.moodle.org/en/Embedded_answers_\(Cloze\)_question_type](https://docs.moodle.org/en/Embedded_answers_(Cloze)_question_type) [Internet; Accedido: 31-05-2026]
- [9] Moodle HQ. *Moodle Documentation: Moodle XML format*. Moodle Docs, 2024. Disponible en: https://docs.moodle.org/en/Moodle_XML_format [Internet; Accedido: 31-05-2026]
- [10] Oracle Corporation. *Java Platform SE 17: javax.xml.parsers — Document Object Model (DOM)*. Oracle Documentation, 2021. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.xml/javax/xml/parsers/package-summary.html> [Internet; Accedido: 31-05-2026]
- [11] Apache Software Foundation. *Apache Maven Project*. Disponible en: <https://maven.apache.org/> [Internet; Accedido: 31-05-2026]
- [12] SonarSource. *SonarQube Cloud — Online Code Review as a Service Tool*. Disponible en: <https://www.sonarsource.com/products/sonarqube/cloud/> [Internet; Accedido: 31-05-2026]
- [13] W3C. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. W3C Recommendation, noviembre 2008. Disponible en: <https://www.w3.org/TR/xml/> [Internet; Accedido: 31-05-2026]
- [14] Eclipse Foundation. *Eclipse IDE for Java Developers*. Disponible en: <https://www.eclipse.org/ide/> [Internet; Accedido: 31-05-2026]
- [15] Apache Software Foundation. *Maven: Introduction to the POM*. Disponible en: <https://maven.apache.org/guides/introduction/introduction-to-the-pom.html> [Internet; Accedido: 31-05-2026]



[16] Conventional Commits. *Conventional Commits — A specification for adding human and machine readable meaning to commit messages.* Disponible en: <https://www.conventionalcommits.org/en/v1.0.0/> [Internet; Accedido: 31-05-2026]